EE 250L: Distributed Systems for the Internet of Things

Fall 2026 - Lab 1

Linux VM and ESP32-S3 Development Setup

Assigned: __8/25/2026____ Submission due: ____9/4/2026_____

Lab outcome: Complete two independent setup checks: (1) demonstrate a working Ubuntu Linux VM with command-line, Git, and Python tools; and (2) demonstrate a working ESP32-S3 toolchain by building, flashing, monitoring, and connecting the board to Wi-Fi.

The two system roles used in this course

This semester uses two distinct system roles. The Ubuntu VM serves as the Linux host/server environment, while the ESP32-S3 serves as the embedded endpoint. ESP-IDF runs on your laptop's native operating system as the development environment used to program the ESP32-S3. The VM and ESP32-S3 do not need to communicate directly in Lab 1; in later labs they may need to.

Environment	Used for	Lab 1 check
Ubuntu Linux VM	Linux commands, Python, Git, servers, networking tools, databases, and other host/server-side infrastructure	Run commands and a Python program; complete a Git commit
ESP32-S3-DevKitC-1	Embedded firmware, Wi-Fi, sensors, actuators, and IoT endpoint behavior	Build, flash, monitor, join Wi-Fi, and obtain an IP address by DHCP

Important: Install and run ESP-IDF on your laptop's native operating system (Windows, macOS, or Linux). You are not required to pass the ESP32-S3 USB device through the VM.

Required hardware and software

- ESP32-S3-DevKitC-1 development board (N16R8 preferred; N8R8 is also supported)
- USB data/power cable compatible with your board; a charge-only cable will not work
- Laptop with sufficient free storage for Ubuntu and ESP-IDF
- Access to a 2.4 GHz Wi-Fi network or phone hotspot whose credentials you control
- A virtualization application appropriate for your laptop architecture (VirtualBox, UTM)

Check 1: Ubuntu Linux VM

At the end of this section, you should be able to start Ubuntu, use its terminal, install software, run Python, and use Git. The VM is the canonical Linux host/server environment for the course.

1. Identify your laptop architecture

Download an Ubuntu image and virtualization application that match the processor architecture of your laptop.

Laptop	Likely architecture	How to check
Apple Silicon Mac (M-series)	ARM64 / aarch64	Terminal: uname -m
Intel Mac	x86_64	Terminal: uname -m
Most Windows PCs	x86_64 / AMD64	PowerShell: $env:PROCESSOR_ARCHITECTURE
Windows on ARM	ARM64	Settings > System > About > System type

2. Create the Ubuntu VM

Follow the instructions below for your laptop architecture. Do not download an AMD64 Ubuntu image for an Apple Silicon Mac, and do not download an ARM64 image for a conventional Intel/AMD PC.

Windows on ARM: VirtualBox support for ARM hosts may differ from conventional Intel/AMD Windows setups. If your Windows laptop reports ARM64, contact the course staff for the recommended VM setup before proceeding.

Apple Silicon Mac (M1, M2, M3, M4, or later): UTM and Ubuntu ARM64

1. Download and install UTM.

UTM download page

2. Download the Ubuntu 24.04 LTS ARM64 Desktop ISO.

Official Ubuntu 24.04 ARM64 image page

Choose the 64-bit ARM (ARMv8/AArch64) desktop image. The filename should contain arm64 and desktop.

3. Create the VM in UTM. Choose Virtualize (not Emulate), select Linux, enable hardware acceleration when offered, and attach the downloaded ARM64 ISO as the boot image.

UTM Ubuntu installation guide

4. Start the VM and install Ubuntu. Follow the Ubuntu installer, create your account, restart when prompted, and eject or detach the installation ISO if UTM boots back into the installer.

Intel/AMD Windows or Intel Mac: VirtualBox and Ubuntu AMD64

1. Download and install Oracle VirtualBox.

VirtualBox downloads

Select the package for your host operating system. Install the matching VirtualBox Extension Pack if the course staff requests it or if you need its additional USB/network features later.

2. Download Ubuntu 24.04 LTS Desktop for AMD64.

Ubuntu Desktop download

The filename should contain amd64 and desktop. Here amd64 means the common x86-64 architecture used by both Intel and AMD processors.

3. Create a new Ubuntu VM in VirtualBox. Select Linux/Ubuntu (64-bit), assign the resources below, create a virtual disk, and attach the downloaded Ubuntu ISO when prompted.

Ubuntu's VirtualBox walkthrough

4. Start the VM and install Ubuntu. Use the guided installation defaults unless the lab staff specifies otherwise. Restart when prompted and remove the ISO from the virtual optical drive if the installer starts again.

Recommended VM resources

- CPU: 4 virtual cores
- Memory: 4-8 GB
- Storage: 40-60 GB recommended (25 GB minimum if space is limited)
- Ubuntu Desktop image matching the VM architecture

Reminder: The VM network and the VM's virtual hardware are not the same as the native host. Commands run in Ubuntu affect the VM unless a shared folder or other explicit connection has been configured.

3. Choose a VM network mode

The two common choices are bridged networking and Network Address Translation (NAT). Either can provide Internet access, but their behavior is different.

Mode	What it does	When to use it
Bridged adapter	Places the VM directly on the same local network as the laptop; the VM normally receives its own address.	Use when another device must initiate a connection to the VM and the physical network permits it.
NAT	Shares the laptop's connection through a virtual router; normally supports outbound connections.	Recommended for Lab 1; usually reliable on managed networks such as USC wireless.

Recommended for Lab 1: Start with NAT. It usually provides outbound Internet access with the fewest problems. Use bridged mode only if needed later when another device must initiate a connection to the VM. For Lab 1, the ESP32-S3 does not need to contact the VM.

4. Verify Internet access

First, test connectivity to a known public IP address:

```
ping -c 4 8.8.8.8
```

Next, test connectivity using a domain name:

```
ping -c 4 www.usc.edu
```

You should see replies followed by a summary reporting transmitted and received packets.

- If both commands work, the VM has outbound Internet access and working DNS resolution.
- If 8.8.8.8 works but www.usc.edu does not, the VM likely has a DNS configuration problem.
- If neither works, confirm that the host laptop has Internet access and check whether the VM is using NAT or bridged networking.

Note: Some networks or websites block ping even when normal Internet access works. If ping fails but sudo apt update succeeds, your VM still has outbound Internet access.

5. Update Ubuntu and install tools

```
sudo apt update
sudo apt upgrade
sudo apt install -y git python3 python3-pip
```

Verify the installations:

```
git --version
python3 --version
```

6. Practice essential Linux commands

Open a terminal and complete the following exercise. Replace <usc_username> with your USC username.

```
pwd
ls
mkdir -p ~/ee250/lab1
cd ~/ee250/lab1
echo '<usc_username>' > student.txt
cp student.txt student-copy.txt
ls -l >> student.txt
mv student-copy.txt backup.txt
ls -l >> student.txt
cat student.txt
rm backup.txt
```

You should be able to understand and explain what each of the above commands does.

7. Run a Python program

Create hello.py using a text editor inside the VM:

```
from platform import platform
from datetime import datetime

name = input("Your name: ")
print(f"Hello, {name}!")
print(f"Time: {datetime.now()}")
print(f"System: {platform()}")
```

Run it:

```
python3 hello.py
```

8. Create a local Git repository

GitHub is encouraged for backup and collaboration, but this lab requires only a local repository. Do not commit passwords, API keys, or Wi-Fi credentials.

```
cd ~/ee250/lab1
git init
git config user.name "Your Name"
git config user.email "your_usc_email@usc.edu"
git add student.txt hello.py
git commit -m "Complete Lab 1 Linux setup"
git status
git log --oneline -1
```

Check 1 complete: Your Ubuntu VM starts, has outbound Internet access, runs the required commands and Python program, and contains a Git commit with student.txt and hello.py.

Evidence: Include one or more screenshots showing (a) student.txt, (b) hello.py running, (c) git status and git log --oneline -1, and (d) the ping output.

Optional Challenge: Figure out how to push your local repository to a private or public repository on GitHub.

Check 2: ESP32-S3 Toolchain and Wi-Fi

At the end of this section, you should be able to build, flash, and monitor ESP32-S3 firmware from your laptop's native operating system. The board will then associate with Wi-Fi and obtain an IPv4 address through DHCP.

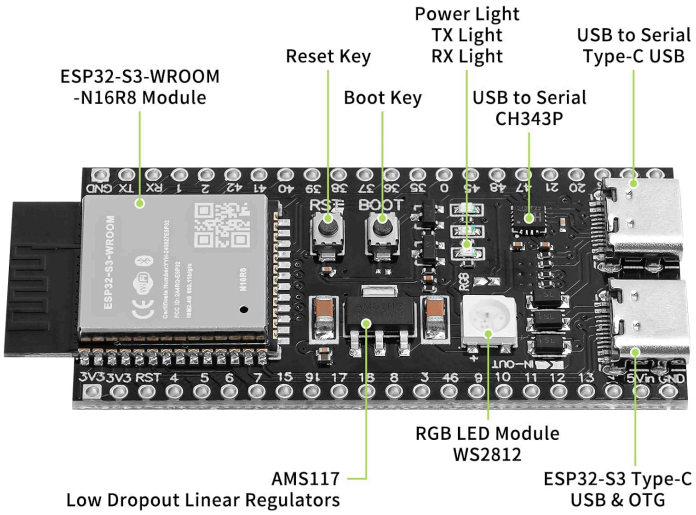

Image source: [YELUFT ESP32-S3-DevKitC-1-N16R8 Development Board User Manual]

1. Inspect the hardware

- Confirm that the board is an ESP32-S3-DevKitC-1, not an older ESP32, ESP32-C3, ESP32-C6, or another model.
- Identify the module marking (for example, N16R8 or N8R8).
- Locate the USB connector used for USB-to-Serial UART programming (it may say COM on the reverse side), the BOOT button, and the RESET/RST button (sometimes labeled EN on similar boards). Your board may or may not look exactly like the photo above depending on the version and manufacturer.
- Use a known USB data cable. If the board powers on but no serial port appears, test a different cable first.

2. Install ESP-IDF on the native host

Follow Espressif's current Getting Started instructions for your native operating system.

Use ESP-IDF **version v6.0.2** even if a newer release exists.

Choose GUI since it provides visual step-by-step guidance through the install process.

Important: Keep the ESP-IDF installation and your ESP-IDF project folders in paths without spaces. On Windows, C:\esp is a safe project location if your user-profile path contains spaces.

ESP32-S3 Getting Started Guide

ESP-IDF setup for Windows

ESP-IDF setup for macOS

ESP-IDF setup for Linux

Serial connection troubleshooting

Open the ESP-IDF terminal or activate the ESP-IDF environment as directed by the installer. Verify:

```
idf.py --version
```

3. Find the ESP32-S3 serial port

Operating system	Typical port name / method
Windows	COM3, COM4, etc.; compare Device Manager before and after connecting the board
macOS (ls /dev/cu.*)	/dev/cu.usbserial-* or /dev/cu.usbmodem*
Linux (ls /dev/tty*)	/dev/ttyUSB0 or /dev/ttyACM0; your user may need serial-port permission

Example output on mac:

```
(venv) bash-3.2$ ls /dev/cu.*
/dev/cu.Bluetooth-Incoming-Port /dev/cu.BoseNC700HP            /dev/cu.debug-console          /dev/cu.usbmodem5C930758231
```

4. Build and flash Hello World

Follow Espressif's official Start a Project instructions for your native host operating system. Copy the hello_world example into your own working directory/workspace folder; do not modify the original example inside the ESP-IDF installation.

Windows: Start a Project from the Command Line

Windows PowerShell note: Espressif's page may show Command Prompt syntax such as %USERPROFILE% and %IDF_PATH%. In PowerShell, use $HOME and $env:IDF_PATH instead.

Linux and macOS: Start a Project from the Command Line

You have to create a new directory – the workspace folder the Espressif docs recommend for storing your ESP-IDF projects (separate from the ESP-IDF tools installation itself); eg for windows: `mkdir -p ~/esp`

After copying the example, open an activated ESP-IDF terminal, change to your copied hello_world project directory, and look at its contents. You should see a CMakeLists.txt file there and a sub-directory called main. If you look inside the main component sub-directory, you should see another CMakeLists.txt file there and the c file containing the hello world program.

Now go back up to the project folder (named something like ~/esp/hello_world/) and run:

```
idf.py set-target esp32s3
idf.py build
idf.py -p PORT flash monitor
```

Run idf.py set-target esp32s3 when you first create or copy each ESP-IDF project. You normally do not need to run it before every build unless you need to change or reset the project target.

Replace PORT with the port found above (e.g. COM3).

Some installations detect it automatically, in which case just typing “idf.py flash monitor” may be sufficient. Take a screenshot. Exit the monitor (GUI) with Ctrl+].

Example output on mac:

```
Hello world!
This is esp32s3 chip with 2 CPU core(s), WiFi/BLE, silicon revision v0.2, 2MB external flash
Minimum free heap size: 401084 bytes
Restarting in 10 seconds...
Restarting in 9 seconds...
Restarting in 8 seconds...
Restarting in 7 seconds...
Restarting in 6 seconds...
Restarting in 5 seconds...
Restarting in 4 seconds...
```

5. Modify, rebuild, and reflash

Modify the program (you can use nano in Mac/Linux or edit from windows command line/Powershell to add the line) in hello_world_main.c so the serial monitor prints your USC username (for example, add a line such as printf("USC username: your_username\n");).

Build, flash, and monitor again. The changed output is evidence that you edited and deployed your own firmware rather than only running a factory or prebuilt image. Take a screenshot.

Example output on mac:

```
Hello world!
USC username: 123456789
This is esp32s3 chip with 2 CPU core(s), WiFi/BLE, silicon revision v0.2, 2MB external flash
Minimum free heap size: 401084 bytes
Restarting in 10 seconds...
Restarting in 9 seconds...
Restarting in 8 seconds...
Restarting in 7 seconds...
Restarting in 6 seconds...
Restarting in 5 seconds...
Restarting in 4 seconds...
Restarting in 3 seconds...
Restarting in 2 seconds...
Restarting in 1 seconds...
```

6. Connect the ESP32-S3 to Wi-Fi

Create a new local project folder called ~/esp/hello_wifi and first copy over everything from the hello_world folder. We will replace key pieces here to flesh out the new project.

Here is how you copy the project folder over in a command line. Run the following from the directory containing hello_world (i.e. ~/esp); make sure hello_wifi does not already exist before typing one of these, as appropriate:

On Mac/Linux this is done from the ~/esp folder in terminal by typing “cp -r hello_world hello_wifi“ ,
On Windows command window by “xcopy hello_world hello_wifi /E /I “
On Windows powershell by “Copy-Item hello_world hello_wifi -Recurse “

Now download the following files located in the course Google drive folder labeled Lab1/ESP32WiFi and put/move them in the appropriate folders of your project:

- hello_wifi/CMakeLists.txt should be moved to your project folder (i.e. to ~/esp/hello_wifi/) - this is the project level make file
- hello_wifi/main/CMakeLists.txt should be moved to your project folder’s main subcomponent folder (i.e. to ~/esp/hello_wifi/main)
- hello_wifi/main/hello_wifi.c should be moved to the main subfolder as well (i.e. to ~/esp/hello_wifi/main)

You may keep or delete the earlier c file from hello world in that folder because it will no longer be read during compile (why? because the new CMakeLists.txt doesn’t mention it!)

Modify the lines for your wifi ssid and password:

```
#define WIFI_SSID "YOUR_WIFI_NAME"
#define WIFI_PASS "YOUR_WIFI_PASSWORD"
```

Build it, flash it, and open the serial monitor.

On command line, type:

```
idf.py set-target esp32s3
idf.py build
idf.py -p PORT flash monitor
```

Credential safety: Never put a real Wi-Fi password in a submitted source file, screenshot, Git commit, or public repository. Keep this Wi-Fi project local unless you have removed your real credentials; do not git add, commit, or push the file containing them.

7. Verify association and DHCP

A successful result should show, in the serial monitor, that the board associated with the access point and received an IPv4 address by DHCP (Dynamic Host Configuration Protocol, which assigns IP configuration to devices).

```
I (...) wifi_demo: Connecting to MyWifi...
I (...) wifi_demo: Connected!
I (...) wifi_demo: IP address: 192.168.1.143

Wi-Fi networking is ready for IP communication.
```

This demonstrates that the ESP32-S3 radio and authentication work and that the network assigned the board usable IP configuration. It does not by itself prove Internet reachability; that depends on the upstream network. Screenshot this.

Wi-Fi notes

- The ESP32-S3 uses 2.4 GHz Wi-Fi. Ensure that your hotspot or access point makes a compatible 2.4 GHz network available.
- On an iPhone hotspot, enabling Maximize Compatibility may help. Keep the hotspot screen open during initial connection if needed.
- An IP address such as 192.168.x.x or 10.x.x.x is normal on a private network.

Check 2 complete: The ESP-IDF environment reports its version; the project targets esp32s3; your modified firmware builds, flashes, and appears in the serial monitor; and the board associates with Wi-Fi and prints an IPv4 address assigned by DHCP.

Evidence: Include screenshots showing (a) the original Hello World output, (b) the modified Hello World output containing your USC username, and (c) the Wi-Fi connection with assigned IPv4 address.

Submission

Each student should submit via Brightspace (we will create an assignment submission for this lab) one PDF file containing:

- One or more screenshots showing student.txt, hello.py output, git status and git log --oneline -1 output, and ping output from the Linux VM exercise
- Screenshots showing (1) the ESP32-S3 Hello World output, (2) the modified Hello World output containing your USC username, and (3) the Wi-Fi connection with assigned IPv4 address
- **An explicit acknowledgment of all assistance received, including AI tools, people, and websites, with a brief description of what help was provided**

Rubric

Points	Check	Evidence
5	Submission for part 1	Screenshots showing completion of all elements for part 1 (VM setup and testing). Acknowledgment with description of assistance received, including from AI or other sources.
5	Submission for part 2 (same file as part 1)	Screenshots showing completion of all elements for part 2 (ESP32 setup and testing). Acknowledgment with description of assistance received, including from AI or other sources.
10	**Total**	

Troubleshooting

Q: The VM has no Internet access.

A: Use NAT mode, especially on USC wireless. Confirm that the native host itself has Internet access, then restart the VM's network connection. Remember that some networks block ping; successful sudo apt update is also evidence of outbound Internet access. If bridged mode was selected, verify that the physical network permits a separately addressed VM.

Q: `brew install --cask eim-gui` fails with "Refusing to load cask ... from untrusted tap."

A: This is expected behavior on newer versions of Homebrew, which require explicit confirmation before installing from a third-party tap. Run `brew trust espressif/eim`, then retry `brew install --cask eim-gui`. This is a one-time step per tap, once trusted, future installs from espressif/eim will proceed normally.

Q: The ESP32-S3 powers on but no serial port appears.

A: Use a known **data-capable** USB cable, try a different USB port, inspect Device Manager or /dev before and after connection, and install the appropriate driver if required.

Q: idf.py selects the wrong chip.

A: From the project directory run idf.py set-target esp32s3, then rebuild.

Q: The build says a component such as esp_wifi is missing.

A: Use the supplied Wi-Fi starter project. The CMakeLists.txt must declare all the components it uses.

Q: Flashing waits for a connection or times out.

A: Confirm PORT, close other serial-monitor programs, and try manual download mode: hold BOOT, tap RESET, release BOOT, then flash again.

Q: The serial monitor shows unreadable text.

A: Use idf.py monitor with the project configuration rather than an arbitrary serial-terminal baud rate. Reset the board after the monitor opens.

Q: Wi-Fi authentication repeatedly fails.

A: Re-enter the SSID and password, confirm 2.4 GHz compatibility, and try the course lab network or a phone hotspot. Do not post the password when requesting help.

Q: The ESP32 gets an IP address but cannot communicate with the VM.

A: That is not required in Lab 1. NAT, bridged networking, firewall rules, and client isolation affect direct reachability. These issues will be addressed when device-to-host networking is introduced.

Q: I get an `idf.py: command not found` error on macOS or Linux.

A: You need to set up the ESP-IDF environment variables in your current shell session.
Run: '. $HOME/esp/esp-idf/export.sh ' (without quotes)
or use the equivalent path to where you installed ESP-IDF. Note the space between the first dot and the script path. You must run this activation script each time you open a new terminal session to work with ESP-IDF, unless you configure a shell alias or other shortcut as described in the Espressif setup guide.

Optional learning resources

- Linux command-line introduction: https://linuxcommand.org/lc3_learning_the_shell.php
- Learn Git Branching: https://learngitbranching.js.org/
- Python tutorial: https://docs.python.org/3/tutorial/
- ESP-IDF examples: https://github.com/espressif/esp-idf/tree/master/examples
- Windows Powershell: https://learn.microsoft.com/en-us/powershell/scripting/overview
- Windows Command Line: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands